

Trabajo Práctico N° 7

Concurrencia

Lectura: Sebesta Cap. 13; Scott Cap. 13.2.3

Nota: Para los ejercicios de Go, pueden utilizar el intérprete disponible en: <https://go.dev/play/>.

Conceptos

1. Describa los niveles de concurrencia que puede contener un programa. ¿Cuáles son los que nos interesan desde el punto de vista de Lenguajes de Programación?
2. ¿Cuáles son las ventajas y desventajas de que un lenguaje presente constructores para dar soporte a la concurrencia de forma nativa frente a que lo haga mediante librerías?
3. Indique cuáles son los principales mecanismos que se pueden utilizar para definir la ejecución concurrente de un bloque de código. Explique brevemente el funcionamiento de cada uno de estos mecanismos y analice sus ventajas y desventajas.
4. Defina sincronización competitiva y sincronización cooperativa, mostrando ejemplos de cada una.
5. Indique cuáles son los principales mecanismos de sincronización. Explique brevemente el funcionamiento de cada uno de estos mecanismos y analice sus ventajas y desventajas.
6. ¿Tendría sentido que un lenguaje funcional brinde mecanismos de sincronización competitiva? ¿Y de sincronización cooperativa? Justifique.

Casos de Estudio e Investigación

7. Describa y compare las formas en las que Java y Ada permiten definir nuevas tareas.
8. Desde un punto de vista conceptual, ¿qué mecanismos de sincronización utiliza Java? ¿Y Ada? Justifique sus respuestas.
9. Compare la forma en que Java y Go incorporan mecanismos para concurrencia.
10. Compare el hecho de modificar una variable atómica con el hecho de utilizar (en Java) un bloque synchronized dentro del cual se modifica una variable.
11. ¿Podría simular los mecanismos de sincronización provistos por Go mediante los métodos wait(), notify() y notifyAll() de Java? Justifique su respuesta.
12. Describa de qué manera pueden utilizar los channels, waitGroups, Mutex de Go para sincronización cooperativa y/o competitiva entre tareas.
13. Explique la diferencia entre un channel sin buffer (unbuffered channel) y un channel con buffer (buffered channel) en Go. Describa cómo el buffering afecta el comportamiento de las operaciones de envío y recepción.
14. El siguiente código ([enlace](#)) en Go intenta enviar un mensaje de una goroutine a otra utilizando un channel.
 - a. ¿Qué observa al ejecutar el código provisto? ¿Por qué cree que ocurre esto?
 - b. Descomente la línea `time.Sleep(1 * time.Second)` en la función `main`. ¿Qué cambia en la ejecución? ¿Por qué esta solución no es ideal para una sincronización robusta?
 - c. Modifique el código para usar un `sync.WaitGroup` que asegure que tanto la goroutine emisor como la receptor hayan terminado su ejecución antes de que `main` finalice, sin depender de un `time.Sleep`. Explique cómo `WaitGroup` resuelve el problema de sincronización en este caso.
15. Considere el siguiente código ([enlace](#)) en Go donde múltiples goroutines intentan actualizar un contador compartido.
 - a. Si ejecuta este código varias veces, ¿el valor final del contador es siempre el mismo? ¿Coincide con el valor esperado (`numGoroutines×1000`)? ¿Por qué?
 - b. Modifique el código para utilizar un `sync.Mutex` para proteger el acceso a la variable contador y asegurar que la operación de incremento sea atómica. Con esta modificación ¿el valor final del contador es siempre el esperado?